



Project Report

Intern Name: Ahmed Islam

Department: Artificial Intelligence

Organization: IR Solutions

Designation: AI Intern

Week # 05

Submitted to: Asad Ali

Evaluated by: Asad Ahmed

Submission Date: 08-Aug-2025

Table of Contents

Project Report	1
Theoretical understanding	5
Neural Networks	5
Back Propagation	6
Gradient Decants.....	6
Activation function	6
Weight and Biases.....	6
Loss Function	6
Type of loss Functions.....	6
Artificial Neural Network (ANN)	7
Mathematics Behind ANN	8
Forward Propagation (Making predictions)	8
Activation function (Non-linearity):	8
Backpropagation (Learning by updating weights)	8
Convolutional Neural Network	9
Image	9
Filter (Kernel).....	10
Convolution	10
Padding	10
Valid:	10
Same:	11
Stride.....	11
Pooling (Max Pooling)	11
Flatten	11
Simple Convolutional Neural networks look like this.	11
Recurrent Neural Network (RNN)	12
RNN Unfolding	12
Recurrent Neural Network Architecture.....	12

Hidden State Calculation.....	12
Output Calculation	12
Overall Function.....	13
How does RNN work?	13
Updating the Hidden State in RNNs	13
Backpropagation Through Time (BPTT) in RNNs.....	13
From output to hidden state.....	14
From hidden state to previous hidden state (recurrence).....	14
Gradients for Recurrent weights	14
Weight Update	14
Questions Related to RNN.....	15
Project Work.....	16
Week1_Day01_Task01: Artificial Neural Networks from Scratch using NumPy and Python	16
Objective.....	16
Methodology.....	16
Week1_Day01_Task02: Customer Churn Prediction using ANN	17
Objective.....	17
Methodology.....	17
Week5_Day2_Task1: Image Representation and Manual Convolution	19
Objective.....	19
Methodology.....	19
Week5_Day2_Task2: Handwritten Digit Classification using CNN with TensorFlow/Keras	22
Objective.....	22
Methodology.....	22
Week5_day3_Task1: Recurrent Neural Network (RNN) for IMDb Sentiment Classification	23
Objective.....	23
Methodology.....	23

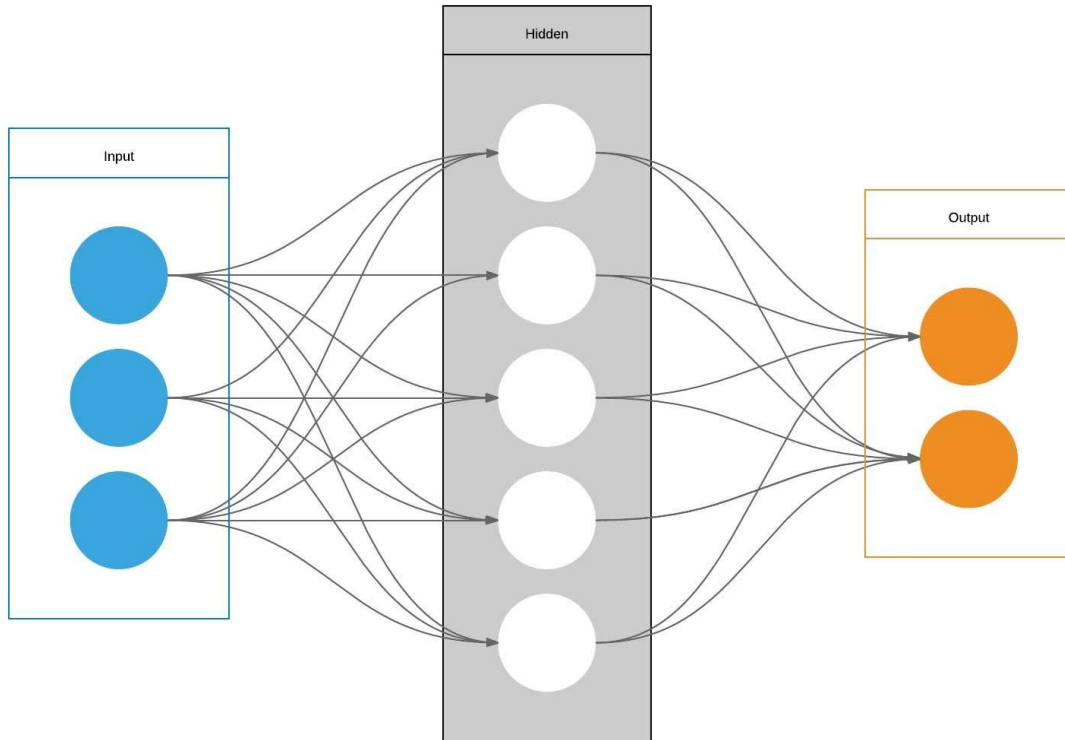
Challenges and Approaches	24
Challenge	24
Solution	24
Outcome	24
Results and Analysis	25
Week5_Day1_Task1 Result	25
Week5_Day1_Task2 Results	25
Week5_Day2_Task1_Results	29
Week5_Day3_Task1_Results	30
LSTM Result	31
Conclusion.....	33

Theoretical understanding

Neural Networks

A neural network consists of neurons, these neurons receive inputs, process them, and produce an output. They are organized into distinct layers: an Input Layer that receives the data, several Hidden Layers that process this data, and an Output Layer that provides the final decision or prediction. After making the prediction a **loss function** calculate how far off the prediction was from the actual result, and the primary goal of training becomes minimizing this "**loss**" or error.

The adjustable parameters within these neurons are called **weights** and **biases**.



$$\hat{y} = \sigma(w^T x + b)$$

This equation represents how a neural network processes the input data at each layer

Back Propagation

Once the error or loss is determined, backpropagation helps adjust the **weights** and **biases** to reduce this error. It acts as a feedback mechanism.

Gradient Decants

Gradient decent are used to adjust the weight and biases efficiently.

Activation function

Used for introducing non-linearity in the neural network. This function decides whether a neuron should be activated based on the weighted sum of its inputs and a bias.

Without activation functions, neural networks would be restricted to modeling only linear relationships between inputs and outputs.

- **For binary classification:** Use the sigmoid activation function in the output layer. It will squash outputs between 0 and 1, representing probabilities for the two classes.
- **For multi-class classification** Use the softmax activation function in the output layer. It will output probability distributions over all classes.
- **If unsure** Use the ReLU activation function in the hidden layers. ReLU is the most common default activation function and usually a good choice.

Weight and Biases

Weights determine the strength of connections between neurons in different layers.

Biases, on the other hand, are constants added to the weighted sum of inputs before applying an activation function. They allow the network to shift the activation function's output, providing flexibility to fit data that doesn't pass through the origin. The bias adjusts the baseline value before activation.

Loss Function

A loss function is a function that compares the target and predicted output values

$$J(w^T, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Type of loss Functions

In supervised learning, there are two main types of loss functions

1. **Regression Loss Functions** – used in regression neural networks. Ex. Mean Squared Error, Mean Absolute Error
2. **Classification Loss Functions** – used in classification neural networks. Ex. Binary Cross-Entropy, Categorical Cross-Entropy

Binary class Entropy Loss:

$$CE Loss = \frac{1}{n} \sum_{i=1}^N - (y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i))$$

Categorical Cross-Entropy Loss:

$$CE Loss = -\frac{1}{n} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(p_{ij})$$

- **Regression: Mean Squared Error (MSE)**

$$L = \frac{1}{m} \sum_{j=1}^m (y_j - \hat{y}_j)^2$$

Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the biological neural networks in the human brain.

An ANN typically has:

Input Layer → Accepts raw features (e.g., pixel values, measurements, etc.)

Hidden Layers → Process data through weights, biases, and activation functions

Output Layer → Produces the final prediction (e.g., class label or numerical value)

Mathematics Behind ANN

Forward Propagation (Making predictions)

Linear combination of inputs:

For a single neuron:

$$z = \sum_{i=1}^n w_i x_i + b$$

Where:

- x_i = input features
- w_i = weights (learnable parameters)
- b = bias (offset term)
- z = weighted sum

Activation function (Non-linearity):

$$a = f(z)$$

Common functions:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU: $f(z) = \max(0, z)$
- Tanh: $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Backpropagation (Learning by updating weights)

Backpropagation is the process of sending the error backward through the network to update weights and biases so the ANN can learn.

We use Gradient Descent to minimize the loss:

1. Compute the **gradient** (partial derivatives) of loss L with respect to each weight:

$$\frac{\partial L}{\partial w_i}$$

2. Update weights:

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i}$$

Where η = learning rate.

Chain Rule in Calculus is used for backpropagation:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

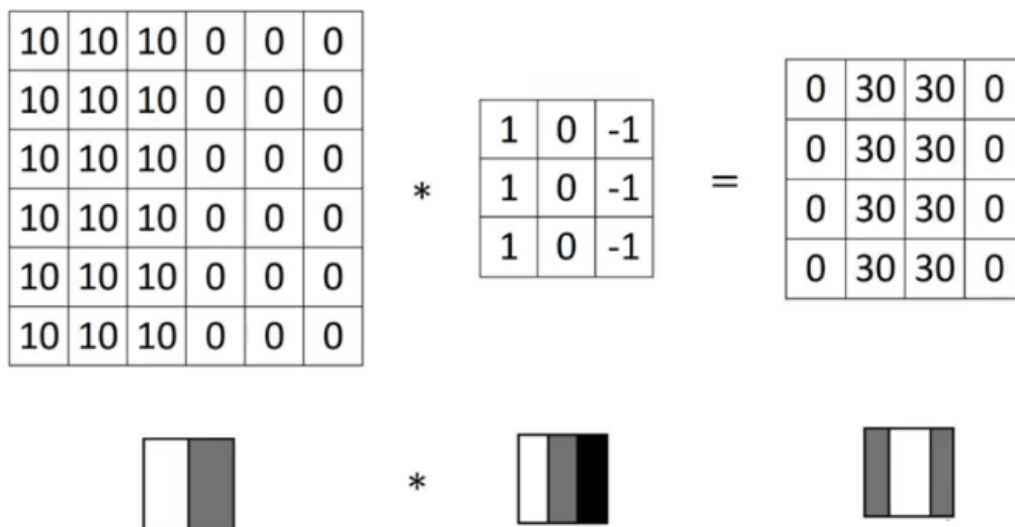
Convolutional Neural Network

CNN (Convolutional Neural Network) is a type of deep learning model that is mainly used for images, but it also works on any data with a grid-like structure.

Think of CNNs as models that look at small parts of an image, detect patterns (like edges, curves, eyes, faces, etc.), and then combine those patterns to understand the full image.

Image

An image is just a grid of numbers (pixel values).



Filter (Kernel)

A small matrix (e.g., 3×3) that slides over the image.

It's like a magnifying glass looking for a specific pattern.

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

Some of the commonly used filters are:

1	0	-1
2	0	-2
1	0	-1

Sobel
filter

3	0	-3
10	0	-10
3	0	-3

Scharr
filter

Convolution

The process of sliding the filter over the image and doing **multiplication & sum**.

Output is a **feature map** (new image showing where that pattern exists).

Padding

- Every time when we apply a convolution operation the size of image shrinks.
- Pixel present in the corner of the image are used only a few number of time as compared to central pixel.

To overcome these issues, we can pad the image with an additional border.

Padding adds extra zero pixels around the image so the output stays the same size.

There are two common choices for padding:

Valid: It means no padding. If we are using valid padding, the output will be $(n-f+1) \times (n-f+1)$

Same: Here, we apply padding so that the output size is the same as the input size, i.e.,

$$n+2p-f+1 = n$$

$$\text{So, } p = (f-1)/2$$

Stride

How many pixels you move the filter at each step.

Stride = 1 → move one pixel at a time (slow but detailed)

Stride = 2 → jump two pixels (faster, less detail)

Pooling (Max Pooling)

Shrinks the size of the feature map.

Keeps important features, removes noise.

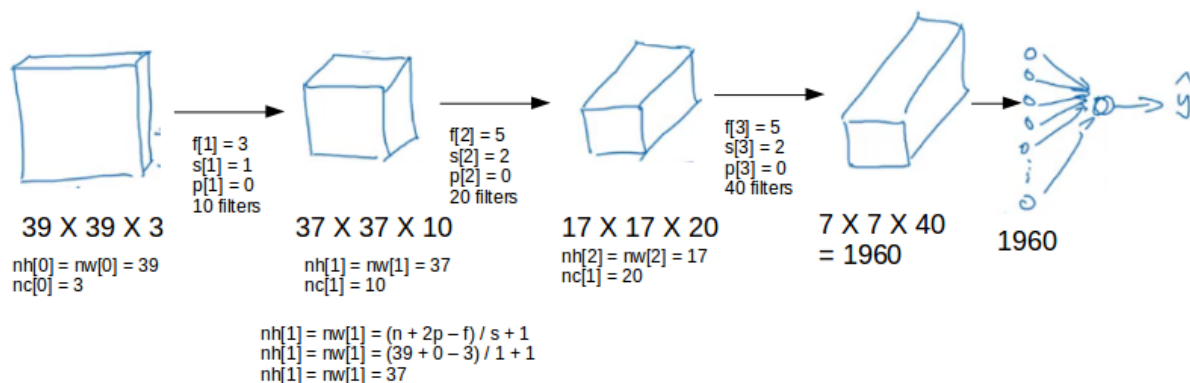
The most useful pooling are:

- Max pooling (from feature map only take the maximum number)
- Average pooling (take the average number)
- Global pooling
 1. Global max
 2. Global average pooling

Flatten

Convert 2d output into a 1D vector before feeding into the final layer.

Simple Convolutional Neural networks look like this.

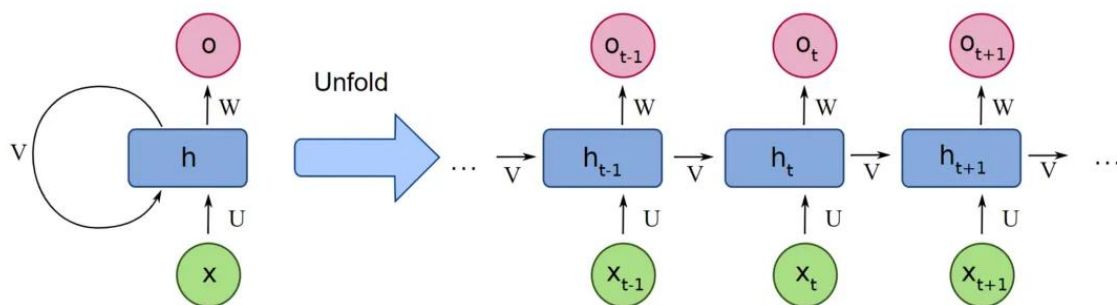


Recurrent Neural Network (RNN)

A Recurrent Neural Network (RNN) is a type of neural network designed to handle sequential data by maintaining a "memory" of previous inputs through recurrent connections.

RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps.



Recurrent Neural Network Architecture

In RNNs the hidden state **h_i** is calculated for every input **x_i** to retain sequential dependencies. The computations follow these core formulas:

Hidden State Calculation

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

- h represents the current hidden state.
- U and W are weight matrices.
- B is the bias.

Output Calculation

$$Y = O(V \cdot h + C)$$

The output **Y** is calculated by applying **O** an activation function to the weighted hidden state where **V** and **C** represent weights and bias.

Overall Function

$$Y=f(X,h,W,U,V,B,C)$$

How does RNN work?

At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps. This memory allows the network to store past knowledge and adapt based on new inputs.

Updating the Hidden State in RNNs

The current hidden state h_t depends on the previous state h_{t-1} and the current input x_t and is calculated using the following relations

State Update:

$$h_t=f(h_{t-1},x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

Activation Function Application

$$h_t=\tanh(W_{hh}\cdot h_{t-1}+W_{xh}\cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

Output Calculation

$$y_t=W_{hy}\cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

Backpropagation Through Time (BPTT) in RNNs

Backpropagation Through Time is the training method for RNNs.

It's just backpropagation (like in ANN) but applied to a network unrolled across time steps.

We now send the error backward

From output to hidden state

$$\delta_{y_t} = \frac{\partial L_t}{\partial y_t}$$
$$\frac{\partial L_t}{\partial W_y} = \delta_{y_t} \cdot h_t^T$$
$$\frac{\partial L_t}{\partial b_y} = \delta_{y_t}$$

From hidden state to previous hidden state (recurrence)

$$\delta_{h_t} = \delta_{y_t} W_y^T + \delta_{h_{t+1}} W_h^T$$

Then apply activation derivative:

$$\delta_{h_t} = \delta_{h_t} \odot f'(z_t)$$

Gradients for Recurrent weights

$$\frac{\partial L}{\partial W_x} + = \delta_{h_t} \cdot x_t^T$$
$$\frac{\partial L}{\partial W_h} + = \delta_{h_t} \cdot h_{t-1}^T$$
$$\frac{\partial L}{\partial b} + = \delta_{h_t}$$

Repeat backward through all-time step until $t=1$

Weight Update

After computing all gradients, update:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

for all shared weights (W_x, W_h, W_y, b, b_y) .

Questions Related to RNN

1. What makes RNNs suitable for sequential data?

Because RNNs maintain a hidden state that is updated at every time step, they can remember previous inputs and capture temporal patterns.

RNNs process inputs one by one and share weights across time, allowing them to generalize across sequences of different lengths.

2. Why do we need to remember past inputs?

In sequential tasks, past inputs provide essential context.

Remembering the past helps the model understand context, patterns, and dependencies over time.

3. Where might a basic RNN fail?

Limitations of basic RNNs:

1. Vanishing/Exploding Gradients

- During training, gradients can become too small or too large → poor learning over long sequences.

2. Short-Term Memory

- Basic RNNs **forget long-term dependencies** (e.g., remembering something from 20 steps ago).

3. Slow to Train on Long Sequences

- Due to sequential processing, they can't be fully parallelized.

That's why we use **LSTM** or **GRU** when long-term memory is needed.

Project Work

Week1_Day01_Task01: **Artificial Neural Networks from Scratch using NumPy and Python**

Objective

The objective of this project was to gain a deep understanding of the internal mechanics of Artificial Neural Networks (ANNs) by building all the core components manually using Python and NumPy. Instead of training a model on a real dataset, the focus was on implementing the mathematical foundation and testing it on dummy data to verify correctness.

Methodology

The implementation was broken down into modular functions, each responsible for a specific task within the ANN workflow:

1. **Parameter Initialization Function**

- A function was created to initialize **weights** and **biases** for each layer.
- Weights were initialized with small random values, while biases were initialized to zeros.
- This ensured symmetry breaking and proper gradient flow during backpropagation.

2. **Activation Functions**

- Implemented activation functions as separate functions for modularity:
 - **Sigmoid**
 - **ReLU**
 - **Relu Derivative**
- Also implemented derivative functions for use in backpropagation.

3. **Forward Propagation Function**

- Function to compute the **linear transformation**:
- Applied the relevant activation function to produce the **activation output (A)**.
- Returned both output and cached values needed for backpropagation.

4. Loss Calculation Function

- Used **Binary Cross-Entropy Loss** for dummy classification output:
- Function took predicted values and true labels, and returned the scalar loss.

5. Backward Propagation Function

- Function implemented to compute gradients of weights and biases using the **chain rule**.
- Gradients were stored and used to update parameters manually.

6. Dummy Data Testing

- Passed a **dummy dataset** with 4 examples, each having 2 features.
- Ran the ANN functions step-by-step (parameter initialization → forward pass → loss calculation → backward pass).
- Verified that each function returned expected numerical outputs, ensuring correctness of implementation.

Week1_Day01_Task02: **Customer Churn Prediction using ANN**

Objective

The aim of this project was to build an Artificial Neural Network (ANN) model to predict customer churn based on historical customer data. The challenge included handling categorical features, preprocessing numerical data, and addressing class imbalance using the SMOTE technique to improve model performance.

Methodology

The workflow was divided into the following steps:

1. Dataset Loading

- Loaded the **Customer Churn** dataset containing both numerical and categorical variables.
- Target variable: **Churn** (1 = Customer Left, 0 = Customer Stayed).

2. Data Preprocessing

- Checked for missing values and handled them appropriately.
- Removed irrelevant columns that do not contribute to prediction (e.g., customer ID).

3. Encoding Categorical Variables

- Used **Label Encoding** to convert object/string features (e.g., gender, payment method, internet service) into numerical form.
- Ensured that all features were in numeric format before feeding into the ANN.

4. Feature Scaling

- Applied **StandardScaler** to normalize features, ensuring all variables had equal importance in model training.

5. Splitting Data

- Divided the dataset into **training** and **testing** sets to evaluate generalization performance.

6. Initial ANN Model Building

- Designed an ANN with:
 - **Input Layer:** Matching the number of features.
 - **Hidden Layers:** Fully connected dense layers with ReLU activation.
 - **Output Layer:** Single neuron with sigmoid activation for binary classification.
- Used **Binary Cross-Entropy** as the loss function.
- Optimized using **Adam** optimizer.

7. Training the Initial Model

- Trained the model on the original dataset.
- Observed lower recall for the minority class due to **class imbalance**.

8. Handling Class Imbalance with SMOTE

- Applied **SMOTE (Synthetic Minority Oversampling Technique)** on the training set to generate synthetic samples for the minority class.
- Rebuilt and retrained the ANN model using the balanced dataset.

9. Evaluation

- Evaluated the model on the test set using **accuracy, precision, recall, F1-score**, and confusion matrix.

- Compared performance before and after SMOTE to assess improvement.

Week5_Day2_Task1: Image Representation and Manual Convolution

Objective

The objective of this lab was to understand how images are stored numerically, how pixel values are represented in both grayscale and RGB formats, and how convolution operations extract meaningful features from images. This exercise involved manually implementing convolution and related operations using NumPy, before using built-in deep learning libraries.

Methodology

The lab was completed in multiple sequential tasks:

1. Grayscale Image Loading and Visualization

- **Loaded** a grayscale image using **PIL** and visualized it using **matplotlib**.
- **Shape & Data Type:**
 - Shape: (*Height, Width*) → No color channel for grayscale images.
 - Data Type: **uint8** (pixel values in range **0–255**).
- **Pixel Representation:** Each pixel stores a single intensity value — darker pixels have lower values, lighter pixels have higher values.

2. RGB Image Loading and Channel Separation

- **Loaded** an RGB image.
- **Separated** the three channels:
 - **R (Red):** Contains intensity of red component.
 - **G (Green):** Contains intensity of green component.
 - **B (Blue):** Contains intensity of blue component.
- **Observation:**
 - Each channel reveals how much of that color contributes to the final image.
 - The green channel often carries the most structural detail because human vision is more sensitive to green.

3. Manual RGB to Grayscale Conversion

- Used the **weighted average formula**:

$$\text{Gray} = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

- **Comparison:**
 - Manual grayscale matched closely with built-in conversion.
 - The weighted formula accounts for human perception, giving more weight to green.

4. Manual Edge Detection using 3×3 Kernel

- Implemented convolution manually using **NumPy** with a **Sobel/Laplacian filter**.
- **Input:** Small grayscale image.
- **Output:** Highlighted edges where intensity changes sharply.
- **Observation:**
 - Kernel detects boundaries in the image by amplifying differences in pixel intensities.

5. Manual Zero Padding

- Implemented zero padding manually:
 - **Padding = 0:** Image size reduces after convolution.
 - **Padding = 1:** Maintains original image size after convolution.
- **Effect:**
 - Padding allows edge pixels to be processed equally as central pixels.
 - Prevents loss of image size during multiple convolutions.

6. Stride Effect

- Applied **stride = 2**:
 - Output size reduced, as the convolution window skips pixels.
- **Mathematical Verification:**
 - Verified output shape matches formula.

7. Testing Multiple Kernels

- Applied different 3×3 kernels:
 - **Identity** → Leaves image unchanged.
 - **Sharpening** → Enhances edges and details.
 - **Gaussian Blur** → Smooths image and reduces noise.
 - **Edge Detector (Horizontal)** → Highlights horizontal lines.
 - **Edge Detector (Vertical)** → Highlights vertical lines.
- **Observation:**
 - Each kernel extracts different aspects of the image — from fine details to smooth textures.

8. General Convolution Function

- Created a reusable function:
 - Accepts **any 2D image** and **3×3 kernel**.
 - Supports **padding** and **stride** options.
- **Testing:**
 - Applied to multiple filters and confirmed correctness.

9. Analysis of Different Kernels

- Convoluting the same image with different kernels gives varied outputs:
 - Edge detection → Highlights boundaries.
 - Blur → Reduces details.
 - Sharpen → Increases contrast of features.
- **Conclusion:**
 - Kernel choice determines what information is extracted from an image.

Week5_Day2_Task2: Handwritten Digit Classification using CNN with TensorFlow/Keras

Objective

The objective of this task is to implement and train a Convolutional Neural Network (CNN) to classify handwritten digits (0–9) from the MNIST dataset using TensorFlow/Keras. This task focuses on replacing the manual NumPy-based implementation with high-level deep learning library functions, while still following the original task structure. The aim is to gain practical experience in applying CNNs to image classification problems efficiently.

Methodology

1. Dataset Loading & Preparation

- Loaded MNIST dataset using `keras.datasets.mnist`.
- Normalized pixel values from `[0, 255]` to `[0, 1]` for faster convergence.
- Reshaped data to include a single channel dimension (28, 28, 1).
- Converted labels to one-hot encoded vectors for multi-class classification.

2. Model Architecture (CNN)

- **Conv2D Layer (32 filters, 3×3 kernel, ReLU)**: Extracted low-level features from the images.
- **MaxPooling2D Layer (2×2)**: Reduced spatial dimensions to improve efficiency.
- **Conv2D Layer (64 filters, 3×3 kernel, ReLU)**: Extracted more complex features.
- **MaxPooling2D Layer (2×2)**: Further reduced spatial size.
- **Flatten Layer**: Converted 2D feature maps into 1D vector for the dense layer.
- **Dense Layer (128 neurons, ReLU)**: Learned high-level feature combinations.
- **Dense Output Layer (10 neurons, Softmax)**: Predicted probabilities for each digit class.

3. Compilation

- **Optimizer**: Adam (adaptive learning rate optimizer).

- **Loss Function:** categorical_crossentropy for multi-class classification.
- **Metric:** Accuracy.

4. Training

- Trained for **5 epochs** with batch size **128**.
- Used validation_split=0.1 to monitor performance on unseen data during training.

5. Evaluation

- Evaluated model performance on the test dataset.
- Displayed sample predictions along with the true labels.

Week5_day3_Task1: Recurrent Neural Network (RNN) for IMDb Sentiment Classification

Objective

To build a sentiment classification model using RNN architecture on the IMDb movie reviews dataset by leveraging TensorFlow and Keras high-level APIs. The task involved text preprocessing, sequence modeling with SimpleRNN, and model evaluation.

Methodology

1. Dataset Loading and Preprocessing

- Loaded the **IMDb sentiment classification dataset** using Keras built-in dataset loader.
- The dataset contains movie reviews labeled as positive or negative.
- Tokenized the text reviews and padded the sequences to a fixed length to ensure uniform input size for the RNN.

2. Model Architecture

- Built a Sequential model with the following layers:
 - **Embedding Layer:** Converts integer tokens to dense vectors of fixed size.

- **SimpleRNN Layer:** With 32 units, processes sequential input data.
- **Dense Layer:** Sigmoid activation for binary classification output.

3. Training

- Compiled the model using binary cross-entropy loss and Adam optimizer.
- Trained the model on the training set with validation on the validation set.

4. Evaluation

- Evaluated the trained model using accuracy metric on both training and validation datasets.

Results

- The SimpleRNN model achieved limited accuracy due to difficulty in capturing long-term dependencies in longer sequences.

Challenges and Approaches

Challenge

The IMDB dataset contains long sequences of text, making it difficult for the SimpleRNN to remember and propagate information through many time steps. This resulted in suboptimal performance and lower accuracy.

Solution

Implemented an **LSTM (Long Short-Term Memory)** model instead of SimpleRNN. LSTM's gated architecture helps retain information over longer sequences, effectively handling the vanishing gradient problem.

Outcome

The LSTM model achieved approximately **88% accuracy**, significantly improving performance on both training and validation datasets.

Results and Analysis

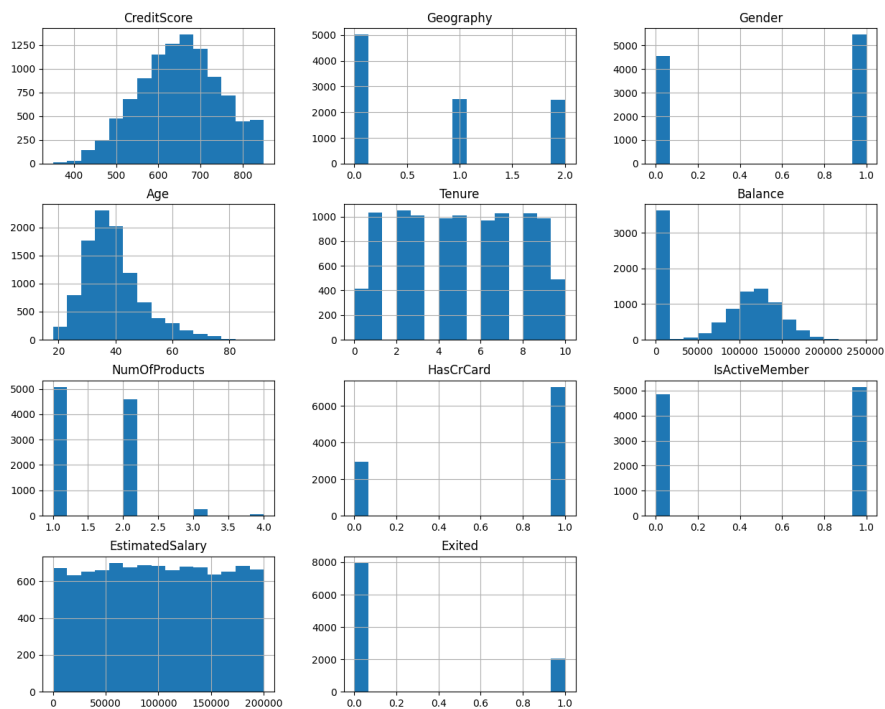
Week5_Day1_Task1 Result

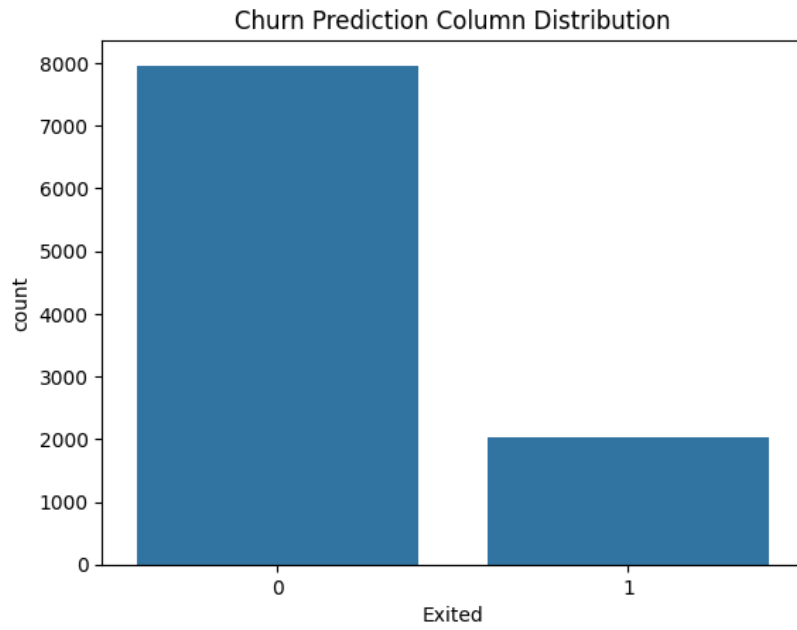
Final Execution - Train and test on AND Gate

```
[ ] X = np.array([[0, 0, 1, 1],  
                 [0, 1, 0, 1]]) # 4 example, each with 2 features  
  
Y = np.array([0, 0, 0, 1]).reshape(1, 4) # AND gate output  
  
parameters = train_neural_network(X, Y, hidden_size=2, iterations=5000, learning_rate=1.0)  
  
preds = predict(X, parameters)  
print("Predictions:", preds)
```

```
⇒ Iteration 0, Cost: 0.6931  
Iteration 1000, Cost: 0.0007  
Iteration 2000, Cost: 0.0003  
Iteration 3000, Cost: 0.0002  
Iteration 4000, Cost: 0.0001  
Predictions: [[0 0 0 1]]
```

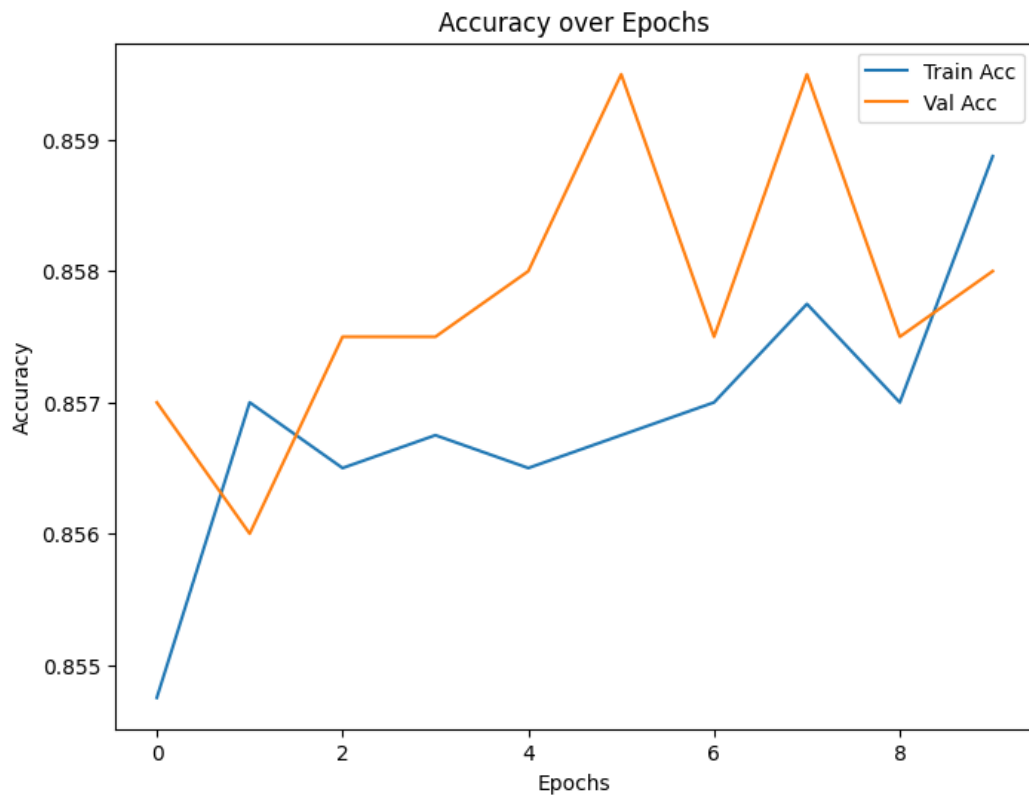
Week5_Day1_Task2 Results

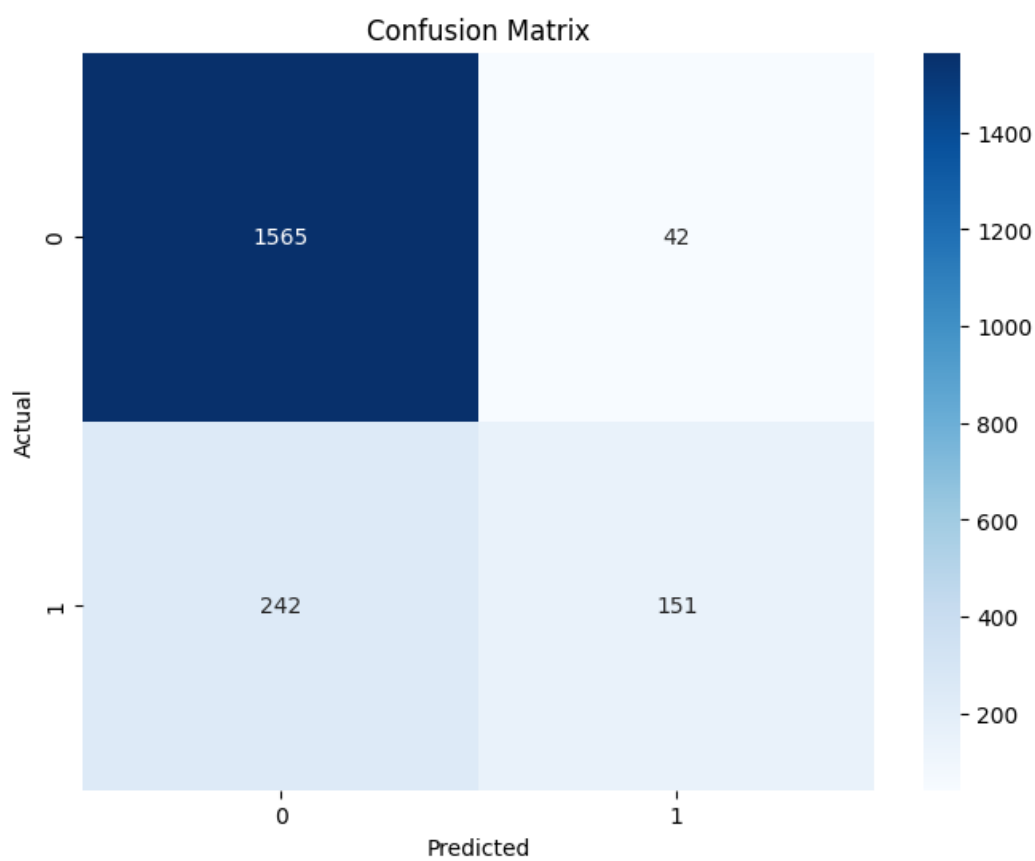
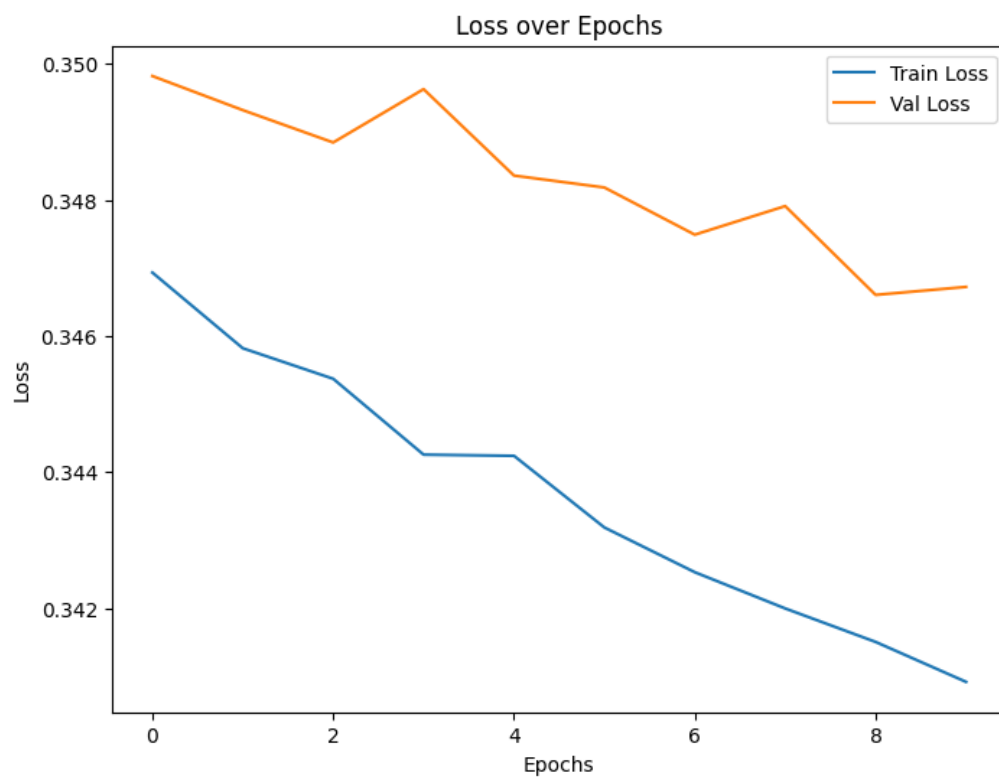




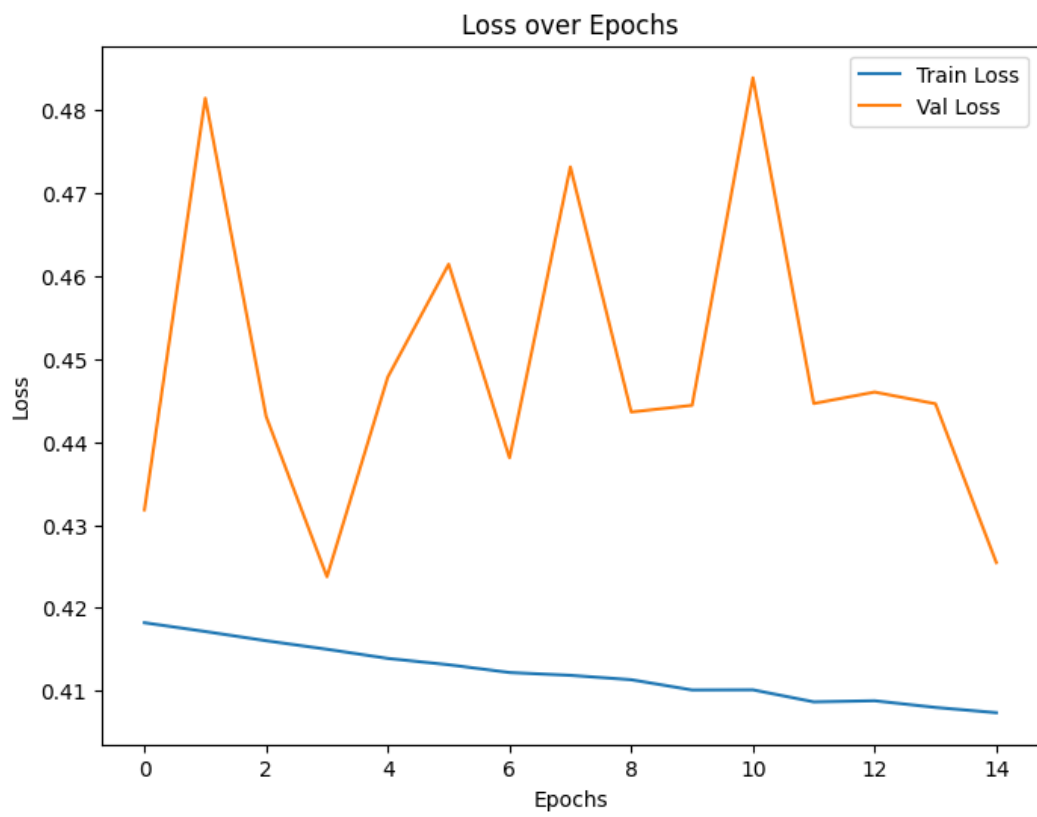
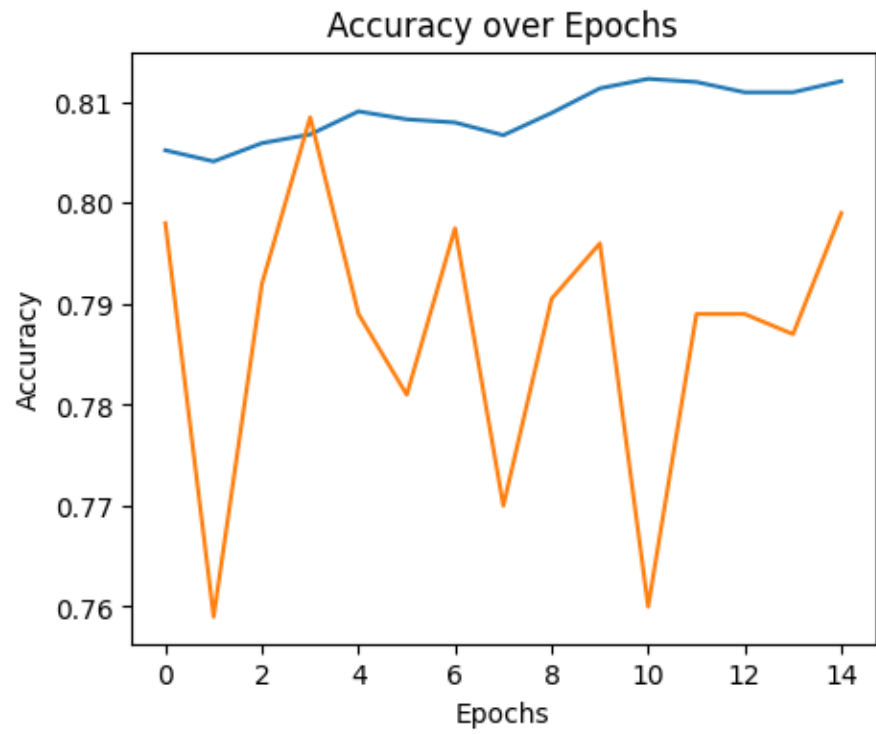
Dataset is Imbalanced

Model train on Imbalanced Dataset

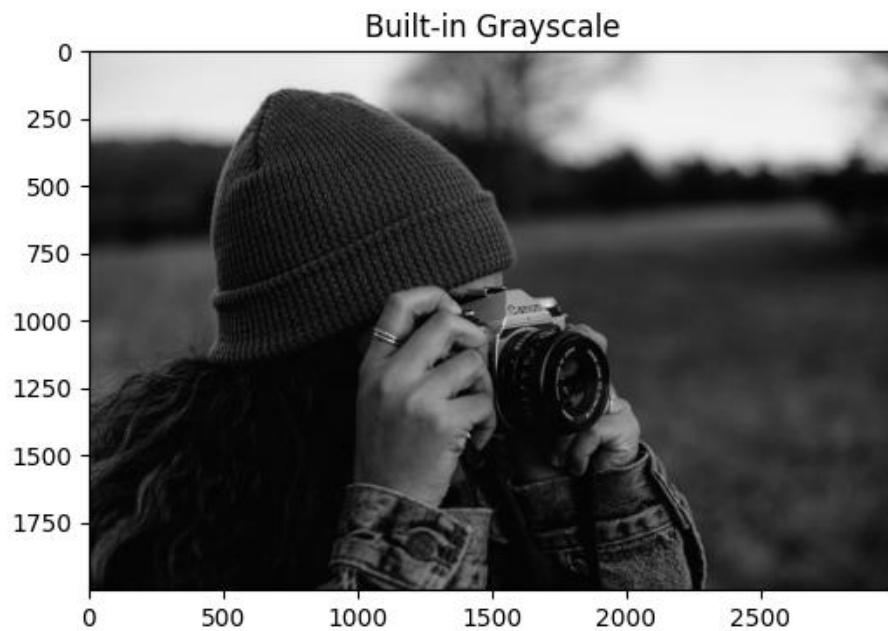
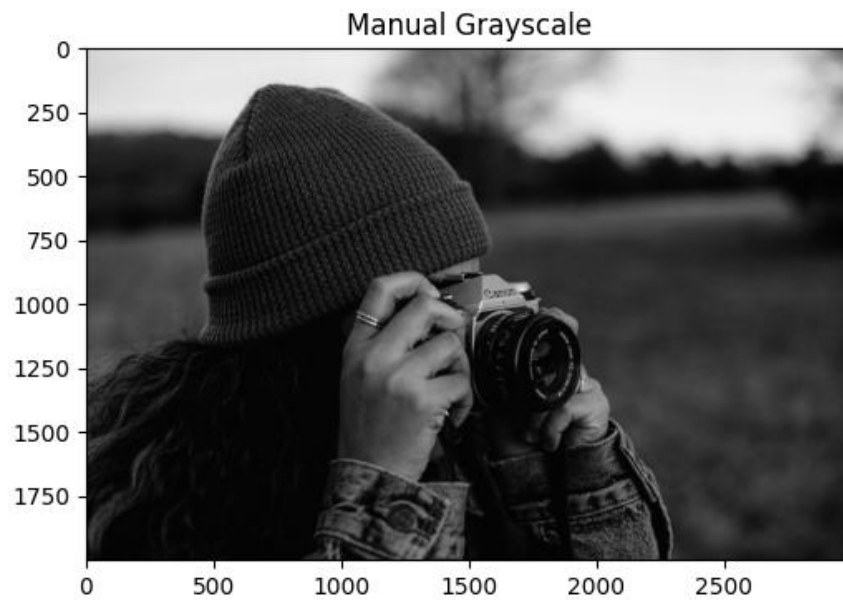
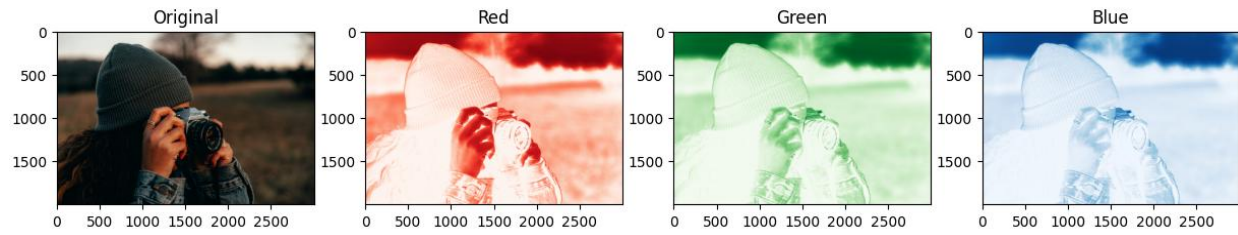


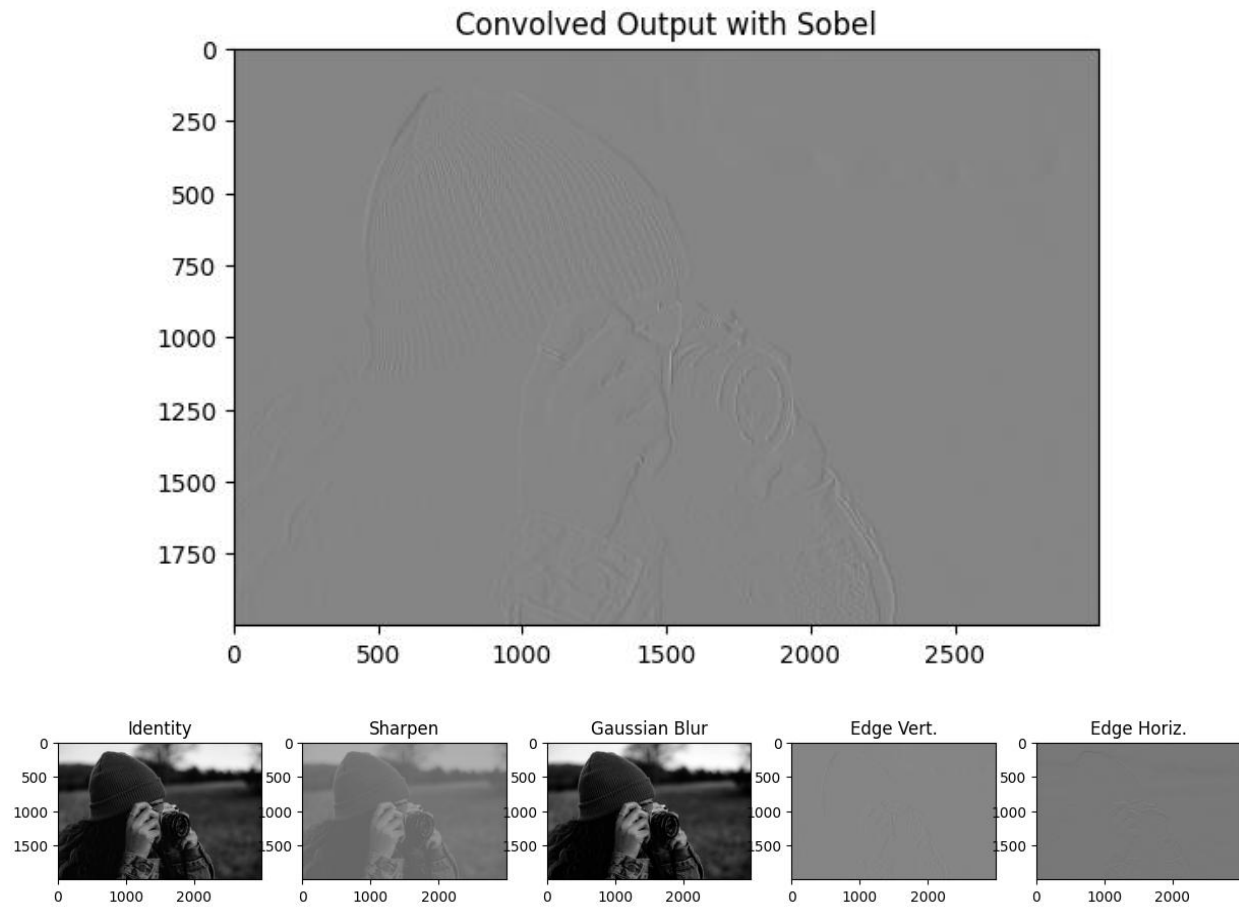


Model train on Balanced Dataset

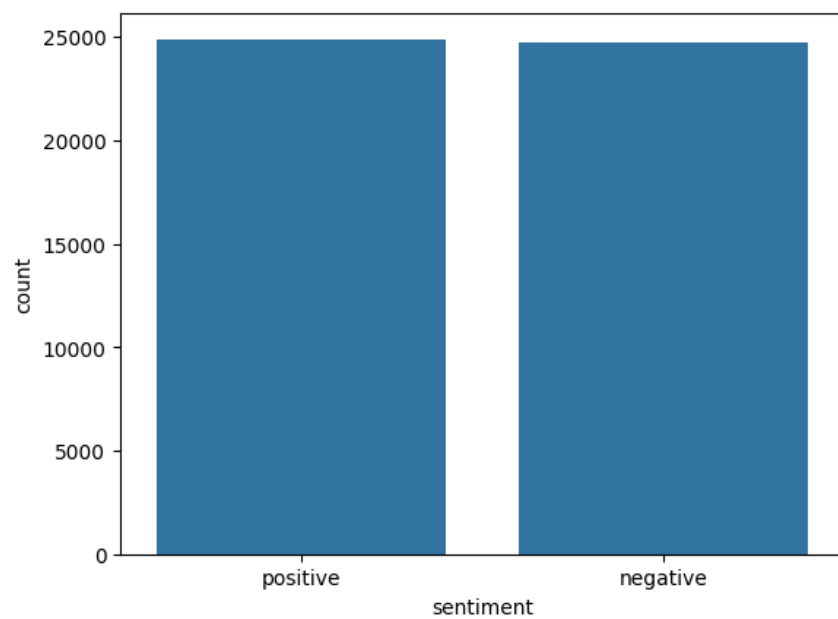


Week5_Day2_Task1_Results





Week5_Day3_Task1_Results



```

Epoch 1/10
310/310 ————— 12s 30ms/step - accuracy: 0.7004 - loss: 0.5527 - val_accuracy: 0.8344 - val_loss: 0.4272
Epoch 2/10
310/310 ————— 16s 21ms/step - accuracy: 0.8828 - loss: 0.3177 - val_accuracy: 0.8638 - val_loss: 0.3458
Epoch 3/10
310/310 ————— 10s 22ms/step - accuracy: 0.9294 - loss: 0.2052 - val_accuracy: 0.8560 - val_loss: 0.3588
Epoch 4/10
310/310 ————— 10s 22ms/step - accuracy: 0.9663 - loss: 0.1114 - val_accuracy: 0.8445 - val_loss: 0.4417
Epoch 5/10
310/310 ————— 10s 21ms/step - accuracy: 0.9831 - loss: 0.0597 - val_accuracy: 0.8474 - val_loss: 0.5304
Epoch 6/10
310/310 ————— 10s 21ms/step - accuracy: 0.9891 - loss: 0.0385 - val_accuracy: 0.8395 - val_loss: 0.6727
Epoch 7/10
205/310 ————— 2s 19ms/step - accuracy: 0.9811 - loss: 0.0600

```

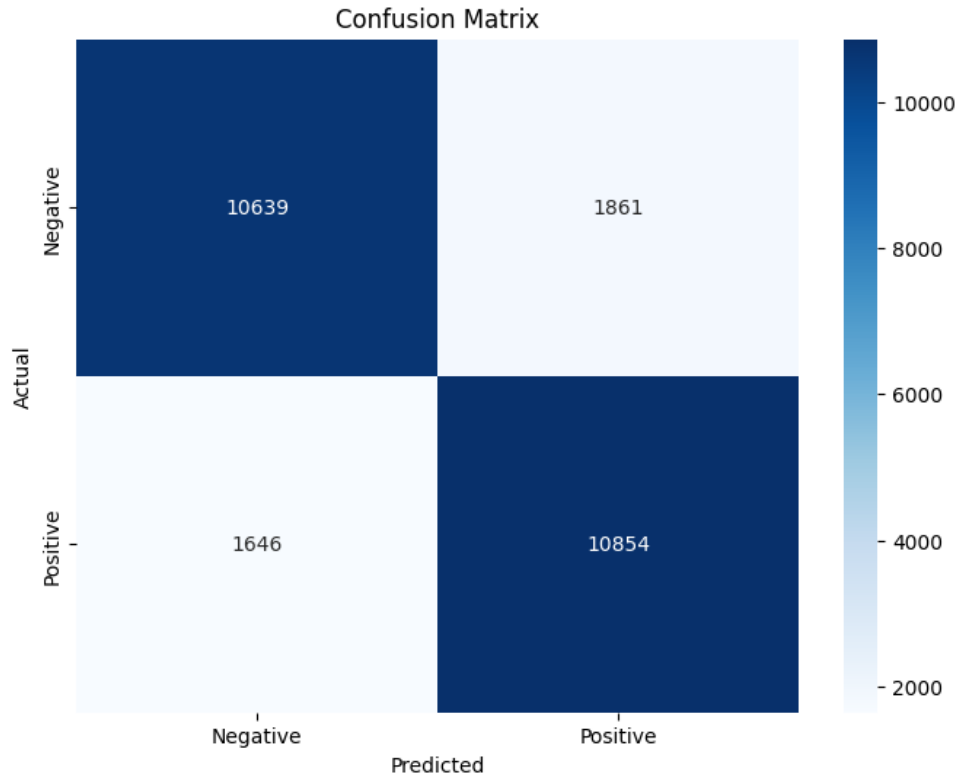
Instead of decreasing the value of loss it increase mean model is overfitting
work well on training dataset but not good on validation dataset.

LSTM Result

```

Epoch 1/10
79/79 ————— 7s 29ms/step - accuracy: 0.6120 - loss: 0.6590 - val_accuracy: 0.7908 - val_loss: 0.4690
Epoch 2/10
79/79 ————— 7s 29ms/step - accuracy: 0.8375 - loss: 0.3821 - val_accuracy: 0.8560 - val_loss: 0.3473
Epoch 3/10
79/79 ————— 2s 22ms/step - accuracy: 0.9019 - loss: 0.2593 - val_accuracy: 0.8850 - val_loss: 0.3040
Epoch 4/10
79/79 ————— 3s 24ms/step - accuracy: 0.9354 - loss: 0.1876 - val_accuracy: 0.8784 - val_loss: 0.2971
Epoch 5/10
79/79 ————— 2s 22ms/step - accuracy: 0.9524 - loss: 0.1493 - val_accuracy: 0.8796 - val_loss: 0.3096
Epoch 6/10
79/79 ————— 2s 22ms/step - accuracy: 0.9594 - loss: 0.1280 - val_accuracy: 0.8560 - val_loss: 0.3325
Epoch 7/10
79/79 ————— 2s 25ms/step - accuracy: 0.9574 - loss: 0.1320 - val_accuracy: 0.8786 - val_loss: 0.3475
Epoch 8/10
79/79 ————— 2s 30ms/step - accuracy: 0.9724 - loss: 0.0916 - val_accuracy: 0.8750 - val_loss: 0.3857
Epoch 9/10
79/79 ————— 2s 24ms/step - accuracy: 0.9765 - loss: 0.0822 - val_accuracy: 0.8618 - val_loss: 0.4511
Epoch 10/10
79/79 ————— 2s 22ms/step - accuracy: 0.9790 - loss: 0.0737 - val_accuracy: 0.8692 - val_loss: 0.4399

```



```
1/1 ————— 0s 36ms/step
Review: This movie was absolutely fantastic, I loved it
Predicted Sentiment: Positive (Confidence: 0.8303)

1/1 ————— 0s 33ms/step
Review: The film was boring and too long. I wouldn't recommend it
Predicted Sentiment: Negative (Confidence: 0.0818)

1/1 ————— 0s 34ms/step
Review: An average movie with some good and some bad parts
Predicted Sentiment: Negative (Confidence: 0.4834)

1/1 ————— 0s 38ms/step
Review: Terrible acting, terrible plot. One of the worst movies ever
Predicted Sentiment: Negative (Confidence: 0.0022)

1/1 ————— 0s 34ms/step
Review: Amazing performances, touching story, and beautiful visuals
Predicted Sentiment: Positive (Confidence: 0.9930)
```


Conclusion

This week, I significantly deepened my understanding of foundational and advanced neural network architectures, including Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).

- I implemented an **ANN from scratch using NumPy**, which strengthened my grasp of the underlying mathematics such as forward propagation, backward propagation, activation functions, and parameter updates.
- Through the CNN lab, I gained practical experience with **image representation and manual convolution operations**, learning how different kernels extract varied features like edges, textures, and shapes. This reinforced my conceptual understanding of how CNNs process spatial data.
- Finally, by working with **RNNs and LSTMs using TensorFlow/Keras**, I explored sequence modeling for natural language processing tasks like sentiment analysis. The challenges faced with SimpleRNN helped me appreciate the importance of advanced units like LSTM in handling long-term dependencies in sequential data.

Overall, this week has built a strong foundation in both theory and practice of neural networks, equipping me with skills to design, implement, and troubleshoot deep learning models across different data types. Moving forward, I plan to enhance my knowledge by exploring more complex architectures and improving model optimization techniques.